Revision 18-19 · Capacity Planning

Interactive revision of lesson 18 and lesson 19: each question as asked, the answer, where I slipped, and the one-line takeaway.

Q1 · Which startup crash did you hit? PARTIAL

Covers: 18

> **QUESTION**
>
> Llama 3 8B on an H100 with vLLM crashes on three days: (1) while loading the model, (2) while sizing the KV cache after setting max_model_len = 1,000,000, (3) during CUDA graph capture. What ran out, and what's the fix for each?

Answer

1. **Loading:** the weights don't fit in GPU memory. Fix: more GPUs (tensor parallel) or lower precision. (For 16 GB on 80 GB, suspect another process occupying the GPU.)
2. **Sizing the cache:** one max-length request can't fit in the pool: 1,000,000 × 128 KiB ≈ **122 GiB** vs ~53 GB. Fix: **lower max_model_len** (e.g. 128k), or grow the pool (FP8 KV, higher budget, more GPUs).
3. **Graph capture:** the headroom outside the 90% budget was too small (graphs take 1–5 GB). Fix: **lower gpu_memory_utilization** (0.9 → 0.85), capture fewer graph sizes, or disable graphs at a speed cost.

Q2 · Chat vs document Q&A on one GPU PARTIAL

Covers: 19

> **QUESTION**
>
> One H100, Llama 3 8B, ~53 GB pool ≈ 400k tokens. Chat: 1,000-token prompts + 500-token replies. Document Q&A: 12,000 tokens of context + 1,000-token replies. How many simultaneous conversations fit, and what does 100 users of each need?

Answer

	Chat	Document Q&A
Tokens per conversation	1,000 + 500 = **1,500**	12,000 + 1,000 = **13,000**
Cache per conversation = tokens × 128 KiB	1,500 × 128 KiB = 192,000 KiB ≈ **0.19 GB**	13,000 × 128 KiB = 1,664,000 KiB ≈ **1.6 GiB**
Fit at once = 400k tokens ÷ tokens per conv	400,000 ÷ 1,500 ≈ **267**	400,000 ÷ 13,000 ≈ **31**
100 users need = 100 × cache per conv	100 × 0.19 ≈ **19 GB**	100 × 1.6 ≈ **160 GB**
GPUs = 100 users ÷ fit per GPU	19 GB < one 53 GB pool → **1 GPU**	⌈100 ÷ 31⌉ = **4 GPUs**

Two ways to get the same GPU count: by **memory** (160 GB ÷ 53 GB per GPU ≈ 4) or by **users** (100 ÷ 31 per GPU ≈ 4). Chat's 19 GB fits inside one GPU's 53 GB pool, so one GPU.

The workload, not the model, sets the hardware bill. Levers: FP8 KV cache (2× tokens → ~2 GPUs for docs), paged allocation.

Q3 · Reserve the max length vs allocate as tokens arrive PARTIAL

Covers: 19, 17

> **QUESTION**
>
> The engine reserves 8,192 tokens per request up front. With a ~400k-token pool, how many requests fit? If conversations average 1,500 tokens and memory is handed out in blocks as tokens arrive, how many fit? Name the technique and its catch.

Answer

1. Reserved: 400k ÷ 8,192 ≈ **49 requests**.
2. Allocated as tokens arrive: 400k ÷ 1,500 ≈ **~270 requests**, ~5.5× more from the same memory.
3. **PagedAttention / paged KV memory** (vLLM): small fixed-size blocks handed out on demand, so there's no fragmentation.
4. **Catch: overcommitment.** The engine bets requests stay short. If many grow long at once, the pool runs out mid-generation and the engine must **preempt** (pause requests, free their blocks, later recompute or swap back): those users see stalls.

Exercise · Simulate a paged KV allocator PARTIAL

Covers: 19, 17

> **QUESTION**
>
> Pool of 64 blocks × 16 tokens; 20 requests (prompt 100, outputs 50–400), one token per step each. Compare reserving 512 tokens up front with paged allocation (a block when needed, preempt the newest request when the pool is dry).

How it's built: track `waiting` (FIFO), `running` (id → [tokens, blocks]) and `free` blocks. Each step: (1) admit while the first waiting request's starting blocks fit; (2) every running request adds a token and, if paged and it spills past its blocks, takes a block or preempts; (3) finished requests return their blocks. Count everything in blocks: need(tokens) = ⌈tokens ÷ 16⌉.

```
want = need(max_len) if policy == "reserve" else need(p)      # admit: BLOCKS, not tokens
...
if policy == "paged" and need(toks) > held:                  # token 113 needs block 8
    if free == 0:
        victim = max(running)                                 # most recently admitted
        free += running.pop(victim)[1]                        # remove it AND free its blocks
        waiting.insert(0, victim)                             # restart later from its prompt
        preempt += 1
        if victim == rid:
            continue
    free -= 1
    held += 1
...
if toks >= p + out:                                           # finished: return blocks
    free += held
```

Policy	Peak running	Preemptions	Steps to finish all
Reserve 512 up front	2	0	2,187
Paged	9	50	1,265

Paged runs 4.5× more requests at once and finishes 1.7× sooner; 50 preemptions are the price.